

Loops in JS

Lec-8

Loops in JS

- Loops can execute a block of code a number of times.
- Loops are handy, if you want to run the same code over and over again, each time with a different value
- JavaScript has following types of loops:
 1. while loop
 2. do while loop
 3. for loop

While loop

- While loops execute a block of code **as long as a specified condition is true**.
- Syntax :**

```
while(condition){
  // code to be executed
}
```

- Suppose we're printing numbers from 1 to 10 using while loop

```
let num = 1
while (num <= 10) {
  console.log(num);
  num+=1
}
// Output: 1 2 3 4 5 6 7 8 9 10
```

- Suppose we're iterating over a array using while loop

```
let arr1 = ['Apple','Oranges','Banana','Jackfruit']
let itevar = 0
while (itevar < arr1.length){
  console.log(arr1[itevar]);
  itevar+=1
}
// Output: Apple Oranges Banana Jackfruit
```

 **Note :**

- If you forget to increase the variable used in the condition, the loop will never end.
- This will crash your browser.

 **Practice Question-1**

A football club's new signing has been given a fitness target by the coach. He needs to complete several training sessions before he's cleared to play in the next match, and the club wants a simple program to track his progress session by session.

Create a variable to track the number of sessions completed, starting at 0 Use a **while loop** that **keeps running until he reaches 5 completed sessions** On each loop iteration, **increase the session count by 1** and print a message showing which session number he just finished Once the loop ends, print a final message saying **"player is fit and ready to play."**

The Output should be in a following way:

Player has completed 1 / 5 training sessions.

Player has completed 2 / 5 training sessions.

Player has completed 3 / 5 training sessions.

Player has completed 4 / 5 training sessions.

Player has completed 5 / 5 training sessions.

The Player is fit and ready to play

 **Practice Question-2**

You're building a quick squad-check tool for your FIFA World Cup 2026 Predictor project. You have a list of player names for one of the teams, and you want to search through it using a while loop to find out if a specific player is part of the squad — **without using any array search methods like includes() or indexOf()**.

Create an array of at least 6 player names - **['m. salah','vini jr','haaland','mbappe','neymar','messi','ronaldo']**

Create a variable to hold the name of the player you're searching for

Use a while loop with an index variable to go through the array one player at a time Inside the loop, check if the current player matches the one you're searching for — if it matches, print that the player was found and stop checking further players

If the loop finishes without finding a match, print a message saying the **Player not available**.

Do While loop

- The **do while** loop is a variant of the while loop.
- This loop will **execute the code block once, before checking if the condition is true**, then it will **repeat the loop as long as the condition is true**.
- The **do while loop at least runs at once**, even if the condition is **false from the start**.
- This is because the **code block is executed before the condition is tested**.
- Do not forget to increase the variable used in the condition, otherwise the loop will never end!
- **Syntax :**

```
do {  
  // code block to be executed  
}  
while (condition);
```

- Suppose we're printing numbers from 1 to 10 using do while loop

```
let variab = 1  
do {  
  console.log(variab);  
  variab+=1  
} while (variab <= 10);
```

- Suppose If we put value of variab as 11 instead of 1 in same code

```
let variab = 11  
do {  
  console.log(variab);  
  variab+=1  
} while (variab <= 10);
```

Above code will print 11 eventough the value stored in variab contradicts the given condition, this is because do while loop runs atleast once before checking the condition.

🔗 Practice Question

A local coffee shop gives out a stamp for every coffee a customer buys, and once they collect enough stamps, they get a free coffee. You're building a quick simulation to see how many coffees a customer buys before earning their free one.

- Create a variable to track the number of stamps collected, starting at 0
- Use a **do while loop** so the customer always buys at least one coffee before any check happens
- Inside the loop, increase the stamp count by 1 and print a message showing the current stamp count after each purchase
- Keep the loop running as long as the stamp count is less than 6
- Once the loop ends, print a message saying the customer has earned a free coffee, along with the total number of coffees it took

For Loop

- The **for** statement creates a loop with 3 optional expressions:
- **Syntax :**

```
for (initiation; condition; incrementation/decrementation)
{
    // code block
}
// initiation: code starts executing from here

// condition : condition to be tested before every execution

// increment/decrement : executes every time after each code block is done executing and changes the value before the
next execution.
```

🔗 Practice Question - 1

You're managing the playlist for a friend's birthday party, and you've got a list of songs queued up. Before the party starts, you want a quick script that announces each song in order as it's about to play, along with its position in the queue.

Create an **array of at least 5 song names (or song + artist pairs, your choice)** Use a **for loop** to go through the array from start to finish On each iteration, print a message showing the song's position in the queue (like "**Song 1: ...**") **along with the song name** After the loop finishes, print a final message saying the **playlist has ended, along with the total number of songs that played**

🔗 Practice Question - 2

You're setting up seating for a wedding reception. There are several tables, and each table has a fixed number of seats. You want a script that prints out a seating label for every single seat at every table, so the venue staff can stick labels on each chair.

Create a **variable for the number of tables (for example, 4)**, Create a **variable for the number of seats per table (for example, 6)**. Use an **outer for loop to go through each table** Inside it, use an **inner for loop to go through each seat at that table** On each inner iteration, print a label like "**Table 1 - Seat 1**", "**Table 1 - Seat 2**", and so on After each table's seats are fully labeled, **print a short separator message** before moving to the next table Once both loops finish, print a final message showing the total number of seats labeled across the whole venue

Break and Continue

- The **break** statement **"jumps out"** of the loops and switches.
- When **break** is encountered in a loop, **the loop terminates immediately**.
- The program **control is transfered to the statements following the loop**.
- **No more loop iterations are executed**.

Practice Question-1

You're checking attendance for a college lecture. **You have a list of registered student names, and the moment you spot a particular student you're looking for, you don't need to keep checking the rest of the list — you can just stop right there and move on.**

Create an **array of at least 6 student names** Create a **variable holding the name of the student you're looking for (make sure it's somewhere in the array)** Use a for loop to go through the array from start to finish **On each iteration, print the name of the student currently being checked As soon as you find a match with the student you're looking for, print a message saying they've been found, along with their position in the list Use the break keyword right after finding the match, so the loop stops checking the remaining students immediately After the loop, print a message confirming attendance check is complete**

- The **continue** statement **skips the current iteration** in a loop.

Practice Question-2

You're going through a **list of grocery items on your shopping app to print a packing slip**, but a **few items in the list are out of stock and marked as null**. You don't want those showing up on the packing slip at all — **you just want to skip over them and continue printing the rest.**

Create an array of at least 7 items where a couple of the entries are null (representing out-of-stock items) and the rest are item names as strings Use a for loop to go through the array from start to finish Inside the loop, check if the current item is null — if it is, use the continue keyword to skip straight to the next iteration without printing anything If the item isn't null, print a message showing its position in the list along with the item name, as if it's being added to the packing slip **After the loop finishes, print a final message saying the packing slip is ready, along with the total number of valid items that were printed (not counting the skipped ones)**

- **break** and **continue** are the only JavaScript statements that can **"jump out of"** a code block.

for ...of Loop

- The **for ...of** loop iterates over the values of an iterable objects such as arrays, strings, etc.
- **Syntax :**

```
for (iterating_variable of iterable){  
  // code block to be executed  
}
```

- Suppose you have an array consisting of some numbers you can print those numbers one by one using for ...of loop

```
const numArr = [56, 5, 96, 102, 89]  
  
for (const i of numArr) {  
  console.log(i);  
}
```

🔗 Practice Question

You're building a quick feature for your FIFA World Cup 2026 Predictor app that announces the qualified teams as they get confirmed, one by one, for a highlight reel shown on screen.

- Create an **array of at least 6 country names** that have qualified for the tournament
- Use a **for...of loop to go through the array**
- On each iteration, **print a message announcing that team as "qualified" (for example, "Brazil has qualified!")**
- Keep a counter variable outside the loop that increases by 1 on every iteration, so you can track how many teams have been announced so far
- After the loop finishes, print a final message showing the total number of qualified teams announced

for ...in Loop

- The **for ...in** loop iterates over the enumerable properties of an object.
- It is typically used for iterating over object keys.
- **Syntax :**

```
for (key in object){  
  // code block to be executed  
}
```

- Suppose you are iterating over a following object

```
const langObj = {  
  js: 'javaScript',  
  cpp: 'C ++',  
  java: 'java',  
  rb: 'ruby'  
}
```

- This can be done in a following way using **for ...in** Loop

```
for(const key in langObj){  
  console.log(`${key} shortcut is ${langObj[key]}`);  
}
```

🔗 Practice Question

You're building a quick feature for your FIFA World Cup 2026 Predictor app You're building a quick player profile card for your FIFA World Cup 2026 Predictor app, and you want a script that prints out every piece of information stored about a player, without having to write out each property by name.

- Create an object representing a player with at least 5 properties (for example: name, country, position, age, goals)
- Use a for...in loop to go through the object
- On each iteration, print the property name along with its value, like "name: Vinicius Jr"
- Keep a counter variable outside the loop that increases by 1 on every iteration, so you can track how many properties were printed in total
- After the loop finishes, print a final message showing the total number of properties that were displayed on the profile card